

JavaScript-09

OOP och funktionell JS

30 september 2026

JavaScript – utan tjafs

Författare: Acke Strömberg

O R I E
R A M M

JS



För en kanin

OOP och funktionell JS

I förra kapitlet lärde vi oss grunderna i hur man bygger klasser och objekt i JavaScript. Det ser väldigt mycket ut som Java eller C#. Men i det här kapitlet ska vi lyfta på motorhuven. JavaScript fungerar nämligen helt annorlunda i bakgrunden. När du förstår vad som döljer sig under ytan kommer du att kunna skriva mycket mer kraftfull kod, och du kommer att förstå varför vi bygger applikationer som vi gör i moderna ramverk som React.

Prototyper – Sanningen om JavaScripts "Klasser"

När vi skriver `class Bil { ... }` i JavaScript så ljugar vi egentligen lite för oss själva. Det finns inga riktiga klasser i JavaScript. Klass-nyckelordet är bara "syntaktiskt socker" – ett snyggare sätt att skriva kod på. Under huven använder JavaScript något som kallas Prototyper. Varje objekt du skapar har en osynlig länk till ett annat objekt, sin prototyp.

Om du letar efter en metod på ett objekt, och den inte finns där, letar JavaScript automatiskt vidare i prototyp-kedjan. Det är på samma sätt som om man letar efter en sak i ett hus: du börjar i rummet där du står. Hittar du den inte där, går du upp på nästa våning och letar vidare. Här är exakt vad som händer när du skriver koden `fido.andas()`.

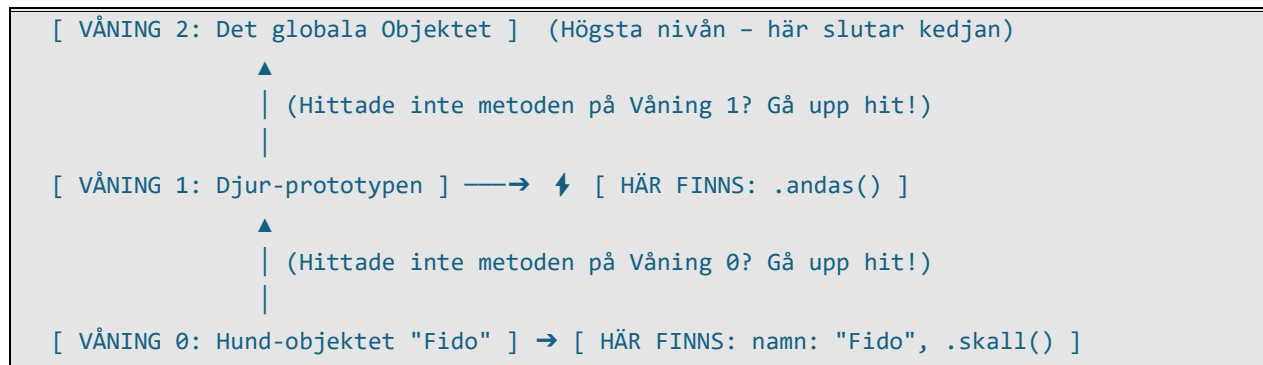


Bild 1 JavaScript letar uppåt i en kedja av objekt tills det hittar metoden.

Vi kan se ett objekts prototyp genom att kika på egenskapen `__proto__`.

```
const fordonsMall = {
  tuta() {
    console.log("Tuuut!");
  }
};

const minBil = {
  varumärke: "Volvo"
};

// Vi länkar ihop minBil med fordonsMall för hand
// (När du använder klasser och "extends" sköter JavaScript detta automatiskt)
minBil.__proto__ = fordonsMall;

minBil.tuta(); // Utskrift: Tuuut!
```

Bild 2 JavaScript hittade inte .tuta() direkt inuti minBil.

Closures (Stängningar) – Funktioner med minne

Nu lämnar vi OOP för en stund och kliver in i den funktionella programmeringens värld. Ett av de absolut mäktigaste koncepten här kallas för ett Closure. En Closure uppstår när en funktion, som ligger inuti en annan funktion, kommer ihåg variablerna från sin "förälder" – även efter att föräldrafunktionen har kört klart och stängts ner.

Tänk på det som att den inre funktionen tar med sig en liten ryggsäck med de variabler som fanns i rummet där den föddes.

Det här är perfekt om man vill skapa variabler som är helt låsta och "privata", så att ingen kan gå in utifrån och förstöra eller ändra på dem av misstag (ett alternativ till # i klasser).

```
function skapaSpelare(namn) {
  let liv = 3; // Den här variabeln är helt "gömd" inuti funktionen

  return {
    visaStatus() {
      console.log(`${namn} har ${liv} liv kvar.`);
    },
    taSkada() {
      liv--;
      console.log(`${namn} tog skada!`);
    }
  };
}

const hjalte = skapaSpelare("Link");

hjalte.taSkada();    // Utskrift: Link tog skada!
hjalte.visaStatus(); // Utskrift: Link har 2 liv kvar.

// Det går INTE att skriva hjalte.liv = 100 utifrån.
// Variabeln "liv" lever säkert i funktionens Closure-ryggsäck!
```

Bild 3 Funktionerna som returneras med ett "Closure".

Higher-Order Functions

Higher-Order functions är egentligen funktioner som hanterar funktioner. I JavaScript säger man att funktioner är **First-Class Citizens**. Det betyder att de kan behandlas precis som vilken datatyp som helst. Du kan spara en funktion i en variabel, skicka med en funktion som ett argument (en så kallad callback), eller låta en funktion skapa och returnera en helt ny funktion.

En funktion som tar emot eller returnerar en annan funktion kallas för en Higher-Order Function. Du använde dessa hela tiden i kapitel 06! `.map()`, `.filter()` och `.reduce()` är klockrena exempel på Higher-Order Functions eftersom de kräver att du skickar in en annan funktion för att de ska fungera. Här skapar vi en egen Higher-Order Function som tar emot en text och en valfri "omvandlings-funktion".

```
// En Higher-Order Function eftersom den tar emot en funktion (stilFunktion)
function formateraMeddelande(text, stilFunktion) {
  return stilFunktion(text);
}

const skrik = (t) => t.toUpperCase() + "!!!";
const viska = (t) => "shhh... " + t.toLowerCase();

console.log(formateraMeddelande("Varning", skrik)); // Utskrift: VARNING!!!
console.log(formateraMeddelande("Hemlighet", viska)); // Utskrift: shhh... hemlighet
```

Bild 4: Skicka med olika funktioner som argument.

Det luriga nyckelordet this

Nyckelordet `this` är känt för att orsaka mycket förvirring. I språk som Java betyder `this` nästan alltid "det här nuvarande objektet i klassen". Men i JavaScript beror värdet på `this` helt och hållet på hur och var funktionen anropas.

- Om du anropar en metod *inuti* ett objekt pekar `this` på objektet.
- Om du anropar en vanlig fristående funktion pekar `this` på det globala fönstret (window i webbläsaren).

Viktigt undantag: Pilfunktioner (Arrow functions `=>`) har inget eget `this`. De lånar istället `this` från koden som ligger precis runt omkring dem.

På bild 5 nedan försöker vi använda en vanlig funktion *inuti* en `setTimeout` (som styrs av webbläsarens Web-API).

```
const användare = {
  namn: "Anna",

  halsa() {
    setTimeout(function() {
      // Det här blir fel/undefined! Eftersom det är en vanlig funktion inuti
      // setTimeout har "this" tappat bort objektet Anna.
      console.log(`Hej, jag heter ${this.namn}`);
    }, 1000);
  }
};

användare.halsa(); // Utskrift efter 1 sek: Hej, jag heter undefined
```

Bild 5 Det vanliga sättet att skriva en funktion tappar bort (`this`) när den körs via asynkront.

Det absolut enklaste sättet att lösa detta i modern JavaScript är att byta till en pilfunktion. Eftersom pilfunktioner saknar ett eget this så lånar den istället this från halsa()-metoden, där this faktiskt är Anna.

```
const anvandareMedArrow = {
  namn: "Anna",
  halsa() {
    setTimeout(() => {
      // Pilfunktionen sparar och lånar "this" från omgivningen!
      console.log(`Hej, jag heter ${this.namn}`);
    }, 1000);
  }
};

anvandareMedArrow.halsa(); // Utskrift efter 1 sek: Hej, jag heter Anna
```

Bild 6 Genom att använda en arrow function (=>) behåller JavaScript rätt referens.
